

Estudo e experimentação sobre o problema de planejamento de horários para cursos universitários utilizando heurísticas paralelas e programação inteira mista

Maria Eduarda Casanova Nascimento¹ e Augusto Vieira Cardona¹.

¹Pontifícia Universidade Católica do Rio Grande do Sul (PUCRS)
Porto Alegre – RS – Brasil

maria.casanova@edu.pucrs.br, acardona@pucrs.br

Resumo. *Este artigo propõe um estudo e experimentação para o problema University Course Timetabling Problem, que é um subproblema do Timetabling Problem. Desde a década de 1960 ele vem sendo pesquisado por inúmeros estudiosos, e por ser um problema de otimização NP-Completo continua sendo um tema relevante para área de pesquisa operacional e de ciência da computação. Além de contextualizar e motivar este assunto, este trabalho apresenta, dois modelos para o problema, um modelo matemático, uma implementação, experimentações e resultados da utilização das técnicas de heurísticas paralelas baseadas em trajetória e programação inteira mista.*

1. Introdução

O problema de agendamento, conhecido como *Timetabling Problem*, é um tipo de problema de otimização combinatória presente no campo de *scheduling*. De acordo com o dicionário, agendar um evento significa "organizá-lo ou planejá-lo para ocorrer em um horário específico". Segundo [Bethel 1956], o agendamento é a fase de controle da produção que classifica as tarefas em ordem de prioridade e, em seguida, as libera no momento adequado e na sequência correta de execução.

O interesse em resolver esse problema vem de longa data como demonstram artigos da década de 1960 [Appleby et al. 1961, Gotlieb 1962, Almond 1969, Kanyazeva and Portugal 1969]. Apesar de muitos pesquisadores terem se dedicado a ele e terem publicado mais de 700 textos nos últimos anos [Bardadym 1996], o *Timetabling Problem* ainda é um problema bastante relevante para a pesquisa operacional e a ciência da computação, de acordo com [Burke et al. 2007].

O *Timetabling Problem* é aplicável a diversas áreas, especialmente instituições de ensino, ambientes empresariais, hospitalares e esportivos [Burke et al. 2004]. Para atrair mais pessoas interessadas nesse problema, atualmente existe uma competição chamada *International Timetabling Competition*, que teve sua quinta edição em 2021 [PATAT 2022]. Ao longo de suas cinco edições, o problema foi abordado em diferentes contextos, como o ensino, com foco em universidades e escolas, e o esportivo.

Neste trabalho, abordaremos o tema do problema de geração de grade de horários dentro das instituições de ensino superior. De acordo com [Carter 1986, Samofalov and Simonenko 1972], a preparação manual de cronogramas em universidades pode levar mais de um mês.

O problema de geração de grade de horários consiste basicamente em coordenar disciplinas, professores, turmas e salas de aula de maneira a garantir que todas as restrições sejam respeitadas. Esse problema pode adotar diferentes formatos, dependendo do contexto em que é aplicado, já que cada instituição possui seus requisitos e condições específicas a serem cumpridos. A literatura existente classificou esses requisitos em obrigatórios (*hard*) e desejáveis (*soft*) [Ahmad et al. 2018], que serão descritos na seção de trabalhos relacionados.

Quando formulado como um problema de programação matemática, os requisitos necessários determinam o conjunto de soluções viáveis e os requisitos desejáveis determinam os critérios de otimalidade [Bardadym 1996]. Esses mesmos requisitos podem ser modelados de outras formas, como por exemplo, através de modelos heurísticos ou métodos de força bruta, mas ainda têm a função de avaliar a qualidade da solução gerada.

Na prática, problemas de otimização costumam ser muito complexos e demandam tempo considerável para serem resolvidos. O paralelismo surge como uma ferramenta natural não apenas para reduzir o tempo de computação, mas também para melhorar a qualidade das soluções obtidas dentro de um prazo definido. Com o aumento do número de processadores em máquinas paralelas, torna-se possível lidar com demandas crescentes de espaço de memória e tempo de processamento, que são essenciais para esses problemas, especialmente o tempo.

Historicamente, os primeiros problemas a serem paralelizados foram: *Knapsack*, *Traveling Salesman Problem*, *Puzzle* e *Queens Problems* [Kindervater and Lenstra 1985, Reeves 1993, Roucairol 1989]. Os autores, na época, estavam apenas testando seus métodos paralelos e não alcançaram muito sucesso em comparação com as versões sequenciais. Logo em seguida, surgiram pesquisas mais bem-sucedidas na tentativa de resolver problemas de otimização usando o paralelismo [Roucairol 1996].

De acordo com [Grama and Kumar 1999], muitos problemas de otimização linear, como os de agendamento, requerem soluções em tempo real, e o processamento paralelo pode ser a única forma de obter um desempenho aceitável. Embora o problema em questão não exija uma resposta em tempo real, o uso de paralelismo ainda proporciona rapidez e eficiência no processo de construção da solução.

Frequentemente estes problemas são formulados como problemas de busca em grafos, a partir de um nodo inicial para vários objetivos possíveis [Johnson et al. 1997, Pearl 1984]. Alguns métodos comumente utilizados em espaço de busca exploratória para problemas de otimização são métodos heurísticos: *Simulated annealing*, *Tabu Search* e algoritmos genéticos.

Este artigo está organizado da seguinte forma: Seção 2 apresenta trabalhos relacionados. Seção 3 apresenta as heurísticas paralelas existentes. Seção 4 apresenta a definição do problema, uma descrição breve dos cenários utilizados nos casos de teste e define os modelos que serão utilizados. Seção 5 apresenta as técnicas utilizadas para resolver o problema proposto. Seção 6 apresenta os resultados obtidos juntamente com a análise. E, por fim, Seção 7 apresenta as conclusões e trabalhos futuros.

2. Trabalhos Relacionados

Os primeiros *software* propostos para resolver tal problema seguiam basicamente dois caminhos, simular a programação de horários feita à mão ou reduzi-lo a um problema matemático [Bardadym 1996]. Ao longo dos anos as pesquisas foram se dividindo em diferentes áreas de estudo para tentar criar ferramentas capazes de solucionar tal problema.

Segundo [de Werra 1985] grafos e redes de fluxos se provaram muito úteis na formulação de soluções para este problema. Um grande argumento para isso é que métodos heurísticos modelados dessa forma são aplicados para solucionar casos reais. E estes são geralmente reduzidos e adaptados a partir de casos mais simples existentes na literatura [Silver et al. 1980].

O problema para criar uma grade de horários válida envolve o agendamento de professores, disciplinas, salas de aula e um fixo número de períodos, de uma forma que nenhum professor, disciplina ou sala, sejam atribuídos mais de uma vez a um período.

Existem muitas variações desse problema, pois os conjuntos de requisitos podem ser alterados, podendo ter versões mais simples do problema com menos restrições, e outras mais complexas mais perto da realidade e com mais restrições. Alguns dos requisitos obrigatórios definidos por [Ahmad et al. 2018] são:

- Não pode haver duas matérias para uma única turma acontecendo simultaneamente
- Professores não podem ter mais de uma turma no mesmo horário
- A mesma sala não deve ser alocada duas vezes para o mesmo horário
- As salas de aula alocadas devem possuir capacidade igual ou superior a capacidade de cada turma

Ao tentar reduzir este problema a outros problemas alguns autores chegaram a conclusões sobre a sua complexidade. [Even et al. 1976] demonstraram em seu artigo que o problema é NP-Completo se cada professor estiver inicialmente indisponível em um subconjunto aleatório dos tempos. [Cooper and Kingston 1996] em seu artigo demonstram que a complexidade da construção de *timetables* em cinco casos específicos do problema são NP-Completo, através de transformações para problemas NP-completos conhecidos como coloração de grafos, problema de empacotamento (*bin packing*) e acoplamento tridimensional (*three-dimensional matching*).

É sabido que esse subgrupo de problemas pode ser abordado através de algoritmos de aproximação, isto é, heurísticas. Por possuir um enorme espaço de busca, que dependendo dos casos se torna impraticável passar por todos os casos para encontrar a solução ótima, se faz necessário chegar em uma solução "boa o suficiente" para cada caso. Por esse motivo inúmeros pesquisadores desenvolveram trabalhos para resolver o problema de geração de grade de horários para universidades utilizando diferentes heurísticas.

Devido a proximidade ao problema de coloração de grafos, alguns autores utilizaram heurísticas baseadas nesse tipo de solução. Outros utilizaram métodos diferentes como programação baseada em restrições, programação inteira, algoritmos evolutivos, otimização baseada em colônia de formigas, *tabu search*, *simulated annealing* e *iterated local search*. Artigos relacionados a cada heurística citada podem ser encontrados no artigo [Lewis 2008].

3. Heurísticas Paralelas

Na prática, problemas de otimização combinatória são geralmente *NP-hard*, complexos e com grande consumo de tempo. Existem duas formas que são majoritariamente utilizadas para solucionar tais problemas, sendo eles os de métodos exatos e os baseados em heurísticas. Métodos exatos garantem uma solução ótima mas raramente são viáveis na prática por consumirem muito tempo. Por outro lado métodos heurísticos garantem soluções quase ótimas ou as melhores possíveis em um tempo razoável.

A maioria das heurísticas são sequenciais, no entanto elas trazem significativa redução em complexidade temporal na busca das soluções. Sendo assim sistemas paralelos, possivelmente com centenas ou milhares de processadores, dão a perspectiva de resolver eficientemente instâncias relativamente grandes de problemas difíceis, sendo uma motivação para o uso do processamento paralelo [Ferreira and Pardalos 1996].

Heurísticas paralelas podem ser divididas em dois subgrupos, as baseadas em trajetória e as baseadas em população [Alba et al. 2012].

Heurísticas baseadas em trajetória são caracterizadas por métodos que aplicam mudanças a uma única solução parcial. A cada passo do caminho no espaço de busca eles procuram uma trajetória até encontrar a solução. Algumas heurísticas bem conhecidas segundo [Alba et al. 2012], pertencem a esse grupo, tais como:

- *Simulated annealing* (SA)
- *Tabu Search* (TS)
- *Iterated local search* (ILS)
- *Greedy randomized adaptive search procedure* (GRASP)

Heurísticas baseadas em população são caracterizadas por começar com uma população e a cada iteração são utilizados métodos estocásticos sobre os indivíduos para evolução desta e gerar novas e melhores populações. Uma função de *fitness* é aplicada a cada iteração para garantir a qualidade das novas instâncias que são geradas. De acordo com [Alba et al. 2012], algumas heurísticas bem conhecidas pertencem a esse grupo, como:

- *Evolutionary algorithms* (EAs)
- *Ant colony optimization* (ACO)
- *Particle swarm optimization* (PSO)
- *Evolutionary Estrategies* (ES)

4. Descrição do Problema

O problema de planejamento de horários é um desafio complexo em que é necessário atribuir recursos disponíveis a objetos em determinados momentos do tempo, de forma a cumprir o maior número possível de restrições desejadas.

Essas restrições podem ser divididas em duas categorias: restrições *hard*, que precisam ser obrigatoriamente satisfeitas para que o problema seja considerado viável (*feasible*), e restrições *soft*, que podem ser satisfeitas se possível. No entanto, quanto mais restrições *soft* forem respeitadas, melhor será o resultado obtido.

Neste trabalho foram usadas como base de estudo os currículos e especificidades da Pontifícia Universidade Católica do Rio Grande do Sul, mais especificamente os da área de computação.

As técnicas apresentadas foram aplicadas em cenários distintos, que variam em termos de número de cursos, salas, professores e suas disponibilidades. Esses cenários foram elaborados com base nos currículos dos cursos de computação, levando em consideração apenas as disciplinas oferecidas presencialmente. Para determinar a quantidade de disciplinas em cada curso, considerou-se uma média de 60 estudantes por nível. Dessa forma, as disciplinas com capacidade inferior a esse valor foram replicadas para atingir esse número. Além disso, as salas foram divididas entre salas comuns, com capacidade para 60 lugares, e laboratórios, com capacidade para 30 lugares. Foram considerados 5 dias e 6 períodos. Deve-se destacar que os professores foram distribuídos de acordo com os códigos das disciplinas, para manter uma proximidade com a realidade.

São apresentados dois modelos para o problema: um mais simples e outro mais complexo. O modelo mais complexo contém requisitos adicionais relacionados às necessidades dos professores. Ambos modelos consistem em vários elementos que os compõem, tais como disciplinas, salas de aula, professores, dias, períodos e as restrições relacionadas entre eles.

Este trabalho se propõe a resolver o problema apresentado de duas formas. Através de métodos exatos utilizando programação inteira mista e através de heurísticas paralelas utilizando algoritmos baseados em trajetória. Com o intuito de comparar os resultados obtidos por ambos, tanto para o modelo simples quanto para o modelo complexo.

4.1. Modelo Simples

O modelo a seguir descreve as restrições *hard* e *soft* utilizadas na definição do problema:

- H1** Todos os períodos associados a cada disciplina devem ser alocados em uma combinação de dia, período e sala distinta
- H2** Disciplinas com mais de um período devem ser consecutivas quando agendadas para o mesmo dia.
- H3** Disciplinas com mais de um período devem ser alocadas no mesmo horário quando alocadas em dias diferentes.
- H4** Disciplinas com mais de um período devem ter um intervalo de pelo menos um dia quando alocadas em dias diferentes.
- H5** Disciplinas com três períodos devem ser alocadas com um intervalo de um dia.
- H6** Disciplinas com mais de um período devem ser alocadas na mesma sala.
- H7** Disciplinas não podem ser alocadas em períodos em que a mesma não está disponível.
- H8** Não devem ser alocadas disciplinas que excedam a capacidade da sala.
- H9** Duas disciplinas não podem ser alocadas na mesma sala quando ocorrem simultaneamente.
- H10** Todas as disciplinas devem ter um professor associado a mesma.
- H11** Professores devem lecionar apenas uma disciplina por período.
- S1** A alocação de disciplinas do mesmo semestre no mesmo horário para cada curso deve ser minimizado.
- S2** Para cada disciplina alocada a uma sala, o seu módulo deve ser o mais próximo da capacidade da mesma.

4.2. Modelo Complexo

O modelo a seguir descreve as restrições *hard* e *soft* utilizadas na definição do problema, levando em consideração as necessidades dos professores:

- H1** Todos os períodos associados a cada disciplina devem ser alocados em uma combinação de dia, período e sala distinta.
- H2** Disciplinas com mais de um período devem ser consecutivas quando agendadas para o mesmo dia.
- H3** Disciplinas com mais de um período devem ser alocadas no mesmo horário quando alocadas em dias diferentes.
- H4** Disciplinas com mais de um período devem ter um intervalo de pelo menos um dia quando alocadas em dias diferentes.
- H5** Disciplinas com três períodos devem ser alocadas com um intervalo de um dia.
- H6** Disciplinas com mais de um período devem ser alocadas na mesma sala.
- H7** Disciplinas não podem ser alocadas em períodos em que a mesma não está disponível.
- H8** Não devem ser alocadas disciplinas que excedam a capacidade da sala.
- H9** Duas disciplinas não podem ser alocadas na mesma sala quando ocorrem simultaneamente.
- H10** Todas as disciplinas devem ter um professor associado a mesma.
- H11** Professores devem lecionar apenas uma disciplina por período.
- H12** Professores devem lecionar apenas disciplinas pré selecionadas.
- H13** Professores não podem possuir uma carga de horário maior que 10 períodos por semana.
- H14** Professores só podem ser alocados em períodos em que o mesmo está disponível.
- S1** A alocação de disciplinas do mesmo semestre no mesmo horário para cada curso deve ser minimizado.
- S2** Para cada disciplina alocada a uma sala, o seu módulo deve ser o mais próximo da capacidade da mesma.
- S3** A quantidade de períodos ociosos entre aulas na agenda dos professores deve ser minimizado.
- S4** A quantidade de dias de trabalho dos professores durante a semana deve ser minimizado.

5. Descrição das técnicas aplicadas ao problema de *Timetabling Problem*

Nesta seção, são descritas as técnicas aplicadas aos cenários mencionados na seção 4: o modelo matemático de programação inteira mista e a ferramenta desenvolvida.

5.1. Modelo matemático

O modelo proposto formula matematicamente os requisitos apresentados anteriormente para o modelo simples. O objetivo é minimizar a alocação de disciplinas do mesmo semestre para o mesmo horário, assim como minimizar a diferença entre o capacidade da sala e o módulo da disciplina. Os termos utilizados na modelagem serão definidos a seguir:

Conjuntos:

$l \in L$ conjunto de disciplinas.

$d \in D$ conjunto de dias.

$h \in H$ conjunto de horas.

$r \in R$ conjunto de salas.

$t \in T$ conjunto de professores.

L_k conjunto de disciplinas de um mesmo semestre, $k = 1, 2, \dots, S$

Parâmetros:

$W_l \in \mathbb{N}^*$ indica o números de períodos por disciplina l .

$N_l \in \mathbb{N}^*$ número máximo de estudantes por disciplina l .

$M_r \in \mathbb{N}^*$ número máximo de estudantes por sala r .

$V_{ldh} \in \{0, 1\}$ indica se a disciplina l tem disponibilidade no dia d e no período h .

$p^{s1} \in \mathbb{N}^*$ penalidade associada a restrição *soft*.

$p^{s2} \in \mathbb{N}^*$ penalidade associada a restrição *soft*.

Variáveis:

$x_{ldhr} \in \{0, 1\}$ indica se a disciplina l esta alocada no dia d , no período h e na sala r .

$y_{lr} \in \{0, 1\}$ indica se a disciplina l esta alocada na sala r .

$z_{ldht} \in \{0, 1\}$ indica se a disciplina l esta alocada no dia d , no período h e com o professor t .

$$\min \quad p^{s1} * \sum_{l \in L_k} \sum_{d \in D} \sum_{h \in H} \sum_{r \in R} x_{ldhr} + p^{s2} * \sum_{l \in L} \sum_{r \in R} y_{lr} * (M_r - N_l) \quad (1)$$

Sujeita a:

$$\sum_{d \in D} \sum_{h \in H} \sum_{r \in R} x_{ldhr} = W_l \quad \forall l \in L \quad (2)$$

$$\sum_{r \in R} x_{ldhr} + \sum_{r \in R} x_{ldh+2r} \leq 1 \quad \forall l \in L, d \in D, h \in H, h = 1, h = 4 \quad (3)$$

$$\sum_{r \in R} y_{lr} \leq 1 \quad \forall l \in L \quad (4)$$

$$\sum_{r \in R} x_{ldhr} \leq V_{ldh} \quad \forall l \in L, d \in D, h \in H \quad (5)$$

$$y_{lr} * N_l \leq M_r \quad \forall l \in L, r \in R \quad (6)$$

$$\sum_{l \in L} x_{ldhr} \leq 1 \quad \forall d \in D, h \in H, r \in R \quad (7)$$

$$\sum_{d \in D} \sum_{h \in H} \sum_{l \in L} z_{ldht} = 1 \quad \forall t \in T \quad (8)$$

$$\sum_{l \in L} z_{ldht} \leq 1 \quad \forall d \in D, h \in H, t \in T \quad (9)$$

5.2. Ferramenta desenvolvida

A ferramenta desenvolvida utiliza heurísticas baseadas em trajetória, e foi desenvolvida em duas etapas. A primeira atende os requisitos do modelo simples e a segunda do modelo complexo.

O Algoritmo proposto, é uma versão paralela da heurística *Greedy Randomized Adaptive Search Procedure* (GRASP) [Feo and Resende 1995], em que uma solução inicial é gerada por um procedimento construtivo guloso. O refinamento da solução é obtido por meio do método de busca local Não Ascendente Randômico (MRNA) [Souza 2022]. O modelo paralelo utilizado foi o mestre-escravo, com troca de mensagens durante a execução do mesmo. O pseudo-código do algoritmo para minimização é apresentado no Algoritmo 1. No algoritmo paralelo o mestre gera a solução inicial e o envia para seus n filhos, estes aplicam o método de busca local e retornam o resultado para o mestre. Por fim o melhor resultado obtido é selecionado. Dessa forma é possível gerar diferentes caminhos de busca simultaneamente, garantindo que potenciais soluções sejam exploradas.

Algoritmo 1: GRASP Paralelo

```
1 mestre
2    $s_0 \leftarrow \text{geraçãoGRASP}()$  ;
3    $s \leftarrow s_0$  ;
4   para todo filho faça
5       envia  $s_0$ ;
6       recebe  $s^*$  ;
7       se  $f(s^*) \leq f(s)$  então
8            $s \leftarrow s^*$ ;
9   retorna  $s$ ;

10 escravo
11   recebe  $s_0$ ;
12    $s \leftarrow \text{MRNA}(s_0, \Delta)$ ;
13   envia  $s$ ;
```

5.2.1. Geração da solução inicial

A heurística GRASP [Feo and Resende 1995] opera de forma iterativa, gerando inicialmente uma solução utilizando um método guloso e aleatório, e em seguida, busca refinar essa solução por meio de buscas locais. O procedimento foi implementado conforme descrito na literatura, com algumas modificações. A fase de construção da solução inicial é feita como segue.

O primeiro passo para gerar a solução inicial é ordenar os elementos candidatos, e estes são inseridos em uma lista restrita de candidatos (LRC). O algoritmo desenvolvido difere do que é proposto na literatura pois não altera a lista de candidatos a cada iteração.

A lista de candidatos contém todas as disciplinas a serem alocadas, ordenadas de acordo com a quantidade de períodos que a mesma possui.

A cada iteração um candidato é retirado da lista, um dia, horário, sala e professor, são selecionados aleatoriamente. E então é feita a tentativa de alocação para todos os períodos, baseando-se nessa combinação. Dessa forma, todos os períodos de cada disciplina são alocados antes de passar para a próxima iteração.

Nessa parte, observa-se uma diferença no desenvolvimento do algoritmo para atender às restrições da primeira e segunda etapa. Na primeira etapa, a escolha do professor leva em consideração apenas a restrição de não poder haver mais de uma disciplina por período na agenda de cada professor. Já na segunda etapa, além das restrição já mencionada, o algoritmo considera as preferências dos professores, tanto em relação às disciplinas que podem lecionar quanto aos horários disponíveis em suas agendas.

5.2.2. MRNA

O Método Randômico Não Ascendente – MRNA é uma variação do método da descida randômica (Random Descent Method – RDM). Ele escolhe, aleatoriamente, um vizinho da solução e o aceita se o seu valor for melhor ou igual ao valor da solução atual

A ideia desse método parte do princípio que é possível desbravar locais planos e, deste modo, sair de um ótimo local. Seu pseudocódigo é descrito no Algoritmo 2. No algoritmo uma solução vizinha, s' , é selecionada em $N(s)$ com um valor de α indo de 1 a 3, para determinar qual tipo de mudança será feita. Esta solução se torna a corrente, se seu valor for melhor ou igual ao valor da solução atual. O método finaliza após $maxIterMRNA$ iterações sem melhora.

Foram criadas três funções de perturbação diferentes, sendo elas: mudança no dia e horário dos períodos da disciplina, mudança da sala para a disciplina e mudança de professor que leciona a disciplina.

Algoritmo 2: MRNA(s , $maxIterMRNA$)

```

1  $iterMRNA \leftarrow 0$  ;
2 enquanto  $iterMRNA < maxIterMRNA$  faça
3    $iterMRNA \leftarrow iterMRNA + 1$  ;
4    $s' \leftarrow seleciona(s, \alpha)$  ;
5   se  $f(s') \leq f(s)$  então
6      $s \leftarrow s'$  ;
7 retorna  $s$  ;
```

6. Resultados

Com o objetivo de avaliar a eficácia e a eficiência do modelo e do algoritmo proposto, foi considerado um conjunto de cenários de testes, com suas características descritas na tabela 1. Ela apresenta a quantidade de elementos utilizados em cada cenário e o nível de restrições referentes aos professores. Com o nível variando de 0 a 2, sendo 0 nenhuma restrição aplicada e 2 um cenário bem restrito. Nos cenários de nível 0 não foram aplicadas restrições aos professores, assim como está definido no modelo simples. Nos cenários

	Cursos	Disciplinas	Salas	Professores	Restrições
Cenário 1	1	61	10	20	0
Cenário 2	1	61	10	20	1
Cenário 3	1	61	10	20	2
Cenário 4	2	117	20	30	0
Cenário 5	2	117	20	30	1
Cenário 6	2	117	20	30	2
Cenário 7	3	164	30	40	0
Cenário 8	3	164	30	40	1
Cenário 9	3	164	30	40	2
Cenário 10	4	233	40	57	0
Cenário 11	4	233	40	57	1
Cenário 12	4	233	40	57	2

Tabela 1. Características dos casos de teste.

de nível 1 e 2 foram aplicadas restrições aos professores condizentes ao que está definido no modelo complexo. Para os de nível 1 os professores possuem uma carga horária limitada, lecionam determinadas disciplinas e uma flexibilidade em relação aos horários disponíveis. Em contra partida para os professores do cenário de nível 2 possuem uma carga horária limitada, lecionam determinadas disciplinas e horários disponíveis bem restritos.

O modelo matemático foi desenvolvido em *Python* utilizando a ferramenta *Pyomo*, e como *solver* o *Gurobi Optimizer*.

O algoritmo foi desenvolvido em C++ em um computador *Apple M2 Max*, contendo 32GB de memória RAM, CPU com 12 cores, sendo 8 de performance e 4 de eficiência, e em um ambiente *macOS Ventura*.

Como há um componente aleatório na solução inicial, as soluções geradas geralmente diferem umas das outras. Por essa razão, em cada cenário, são geradas 10 resultados. Os resultados apresentados são aqueles que geraram a melhor solução após a aplicação das técnicas.

6.1. Resultados do modelo simples

Durante a estudo foram desenvolvidas três diferentes modelos matemáticos para a resolução do problema, o mais recente esta descrito na seção 5.1. As experimentações foram feitas para os cenários de nível 0, utilizando diferentes *solvers*.

A primeira tentativa foi utilizado o CPLEX, porém o mesmo na sua versão gratuita não comportou o tamanho do problema. E sua versão paga possui um valor alto, e havia garantia de sucesso utilizando o mesmo. Foram feitas outras tentativas com *solvers* gratuitos, porém todos demandavam muito tempo de execução e não chegaram a um resultado satisfatório. Por fim foi utilizado o *Gurobi* em sua versão de estudante, o mesmo para os três diferentes modelos desenvolvidos não encontrou um resultado para o problema.

Os resultados obtidos pelo método exato utilizando o modelo matemático de programação

Cenário	Geração inicial	Resultado	Tempo	T_S	T_D	T_O	T_M	μ_S	μ_D
2.8	229300	14100	54,92 seg	4	39	0	0	0,5	2
2. μ	237660	15870	55,03 seg	4	44	0	0	0,6	2
2. σ	14928,88	1307,29	0,41 seg	1,78	2,88	0	0	0,22	2
3.6	201700	15700	48,48 seg	3	43	1	0	0,4	2
3. μ	232840	17630	48,17 seg	7	42	0,5	0	0,8	2
3. σ	25518,72	1116,59	0,63 seg	1,99	6,68	0,53	0	0,25	0,33
5.9	336000	27800	77,16 seg	8	70	2	0	0,5	2
5. μ	363400	30820	77,53 seg	11	79	0,7	0	0,7	3
5. σ	29372,02	1823,79	0,4 seg	2,11	0	1,06	0	0,13	0,15
6.2	374500	28900	71,5722 seg	9	75	1	0	0,6	3
6. μ	360940	31880	71,69 seg	14	77	0,3	0	0,9	3
6. σ	30791,02	2159,63	0,45 seg	3,4	4,08	0,48	0	0,21	0,14
8.6	501400	39300	103,59 seg	14	103	0	0	0,6	3
8. μ	509710	41260	104,04 seg	16	105	0	0	0,7	3
8. σ	31505,08	1359,9	0,58 seg	2,1	2,78	0	0	0,09	0,07
9.2	521500	40200	100,92 seg	16	102	0	0	0,7	3
9. μ	490470	42540	99,56 seg	18	106	0	0	0,7	3
9. σ	29704,85	1885,74	0,78 seg	3,39	5,49	0	0	0,14	0,14
11.4	653400	91200	150,22 seg	26	152	0	60	0,8	3
11. μ	687920	93190	148,92 seg	31	149	0	60	0,9	3
11. σ	40972,48	1785,4	0,63 seg	3,1	5,38	0	0	0,09	0,09
12.2	664600	96300	131,68 seg	32	157	0	60	0,9	3
12. μ	655800	98310	134,35 seg	37	153	0	60	1	3
12. σ	46321,25	3403,09	4,5 seg	4,36	6,03	0,48	0,13	0	0,01

Tabela 2. Resultados usando Heurísticas baseadas em trajetória

inteira mista não foram adequados para resolver o problema proposto pelo artigo. Portanto, os cenários de nível 0 e o modelo simples não foram analisados no contexto desse estudo.

6.2. Resultados do modelo complexo

Os resultados alcançados pela ferramenta desenvolvida utilizando heurísticas baseadas em trajetória para o modelo complexo são apresentados na tabela 2. Foram considerados apenas os cenários com nível de restrição 1 e 2.

Para uma análise mais aprofundada dos resultados, a tabela apresenta uma linha com a melhor solução para o problema, outra com a média das 10 soluções geradas (μ) e a linha seguinte com o desvio padrão da média (σ). Os valores para T_S , T_D , T_O , T_M , μ_S e μ_D são respectivamente: o total de disciplinas alocadas no mesmo horário que pertencem ao mesmo semestre, o total de dias de trabalho de todos os professores, o total de períodos ociosos nas agendas de todos os professores, a diferença total entre o módulo de uma disciplina alocada em relação a capacidade da sala, a média das disciplinas que ocorrem simultaneamente e a média de dias de trabalho de cada professor.

Para realizar o cálculo da função objetivo, levamos em consideração os valores correspondentes às restrições S1, S2, S3 e S4, aplicando as respectivas penalidades de 600, 500, 1000 e 300 para cada valor. É importante ressaltar que a função objetivo nunca atinge zero, uma vez que ela leva em consideração a soma total de dois fatores que não possuem um valor ideal. Um desses fatores é a quantidade de disciplinas alocadas no mesmo horário e que pertencem ao mesmo semestre, existindo casos em que ocorrem disciplinas replicadas. O outro fator é o total de dias de trabalho de todos os professores na semana, não havendo uma média ideal de dias de trabalho para cada professor.

Os resultados na tabela 2 mostram que o algoritmo desenvolvido é capaz de encontrar resultados com bom desempenho em todos os casos. Todos chegaram em valores que satisfazem as quatro restrições *soft* apresentadas. Os conjuntos de dados em todos os seus tamanhos e variações mantêm uma boa uniformidade em relação aos resultados obtidos, exceto ao valor da geração inicial devido ao seu fator aleatório.

Em relação a S1, todos os cenários tiveram uma média igual ou menor a 1. Em relação a S2, com exceção dos cenários 11 e 12, todos os resultados chegaram a zero. Contudo como foi definido na seção 4, as salas possuem a capacidade igual a 30 e 60. E os cenários 11 e 12 possuem 6 disciplinas com capacidade para apenas 20 estudantes. Dessa forma todos cenários chegaram aos valores ideais para essa restrição. Em relação a S3, o algoritmo se mostrou bem eficiente, embora alguns cenários não tenham alcançado o valor ideal. Em relação a S4, todos cenários apresentaram uma boa média de dias trabalhados.

Para todos os cenários o algoritmo foi executado em uma máquina de memória compartilhada com 8 processos. Durante a execução dos testes com o máximo de processos disponíveis, notou-se que ao fazer uso de *hyperthreading* o tempo de execução aumentou significativamente e os resultados mantiveram a mesma média de valores.

O uso do modelo paralelo mestre-escravo trouxe alguns benefícios para a solução como:

- Desempenho aprimorado, tornou-se possível explorar várias opções de solução simultaneamente.
- Maior diversidade de soluções, tornou-se possível explorar independentemente diferentes regiões do espaço de busca.
- Tolerância a falhas, se uma instância de busca falhar ou encontrar uma solução insatisfatória, outras instâncias em execução ainda podem continuar a busca e oferecer resultados válidos, aumentando a robustez e confiabilidade da abordagem.

7. Conclusão

Este trabalho tem como objetivo abordar o desafio de planejar os horários dos cursos universitários na área de computação da PUCRS. Para alcançar esse objetivo, foi proposto e implementado um algoritmo e um modelo matemático específicos. O desempenho do algoritmo foi avaliado em diversos cenários e mostrou resultados satisfatórios, inclusive para conjuntos de dados de diferentes tamanhos, com tempo de processamento polinomial. No entanto, os resultados obtidos ao utilizar o modelo matemático desenvolvido não foram satisfatórios.

Para futuras pesquisas, seria benéfico reexaminar o modelo matemático proposto a fim de obter resultados mais satisfatórios. Quanto ao algoritmo desenvolvido, seria interessante testá-lo em ambientes com maior disponibilidade de recursos de processamento, a

fim de avaliar seu desempenho nessas condições. O mesmo foi desenvolvido pensando em cenários de teste em máquinas distribuídas mas por limitações durante o estudo não foi possível realizá-lo nessas condições. Ademais, seria valioso realizar um estudo comparativo utilizando dados reais em relação a disponibilidade dos professores, a fim de comparar os resultados obtidos pelo algoritmo com o método atualmente utilizado.

Referências

- Ahmad, I. R., Sufahani, S., Ali, M., and Razali, S. N. (2018). A heuristics approach for classroom scheduling using genetic algorithm technique. *Journal of Physics: Conference Series*, 995:012050.
- Alba, E., Luque, G., and Nesmachnow, S. (2012). Parallel metaheuristics: Recent advances and new trends. *International Transactions in Operational Research*, 20:1–48.
- Almond, M. (1969). A University Faculty Timetable. *The Computer Journal*, 12(3):215–217.
- Appleby, J. S., Blake, D. V., and Newman, E. A. (1961). Techniques for Producing School Timetables on a Computer and their Application to other Scheduling Problems. *The Computer Journal*, 3(4):237–245.
- Bardadym, V. A. (1996). Computer-aided school and university timetabling: The new wave. In Burke, E. and Ross, P., editors, *Practice and Theory of Automated Timetabling*, pages 22–45, Berlin, Heidelberg. Springer Berlin Heidelberg.
- Bethel, L. (1956). *Industrial Organization and Management*. McGraw-Hill industrial organization and management series. McGraw-Hill.
- Burke, E., de Werra, D., and Kingston, J. (2004). Applications to timetabling. *Handbook of Graph Theory*, pages 445–474.
- Burke, E., Mccollum, B., Meisels, A., Petrovic, S., and Qu, R. (2007). A graph-based hyper-heuristic for educational timetabling problems. *European Journal of Operational Research*, 176:177–192.
- Carter, M. W. (1986). Or practice—a survey of practical applications of examination timetabling algorithms. *Operations Research*, 34(2):193–202.
- Cooper, T. B. and Kingston, J. H. (1996). The complexity of timetable construction problems. In Burke, E. and Ross, P., editors, *Practice and Theory of Automated Timetabling*, pages 281–295, Berlin, Heidelberg. Springer.
- de Werra, D. (1985). An introduction to timetabling. *European Journal of Operational Research*, 19(2):151–162.
- Even, A. S. S., Itai, A., and Shamir, A. (1976). On the complexity of timetable and multi-commodity flow problems. *Journal of Computing*.
- Feo, T. and Resende, M. (1995). Greedy randomized adaptive search procedures. *Journal of Global Optimization*, 6:109–133.
- Ferreira, A. and Pardalos, P. M. (1996). *SCOOP: Solving Combinatorial Optimization problems in parallel*, pages 1–6. Springer Berlin Heidelberg, Berlin, Heidelberg.
- Gotlieb, C. C. (1962). The construction of class-teacher time-tables. In *IFIP Congress*.

- Grama, A. and Kumar, V. (1999). State of the art in parallel search techniques for discrete optimization problems. *IEEE Transactions on Knowledge and Data Engineering*, 11(1):28–35.
- Johnson, E. L., Nemhauser, G., and Savelsbergh, M. (1997). Progress in integer programming: An exposition. *INFORMS Journal on Computing*, submitted.
- Kanyazeva and Portugal (1969). Computer-aided university department timetable construction. Institute for Cybernetics(Russian), Kiev.
- Kindervater, G. and Lenstra, J. (1985). Parallel algorithms. *Combinatorial Optimization: Annotated Bibliographies*, pages 106–128.
- Lewis, R. (2008). A survey of metaheuristic-based techniques for university timetabling problems. *OR Spectrum*.
- PATAT (2022). Community support. <https://www.patatconference.org/communityService.html>. Accessed: 2023-11-20.
- Pearl, J. (1984). *Heuristics: intelligent search strategies for computer problem solving*. Addison-Wesley Longman Publishing Co., Inc.
- Reeves, C. R. (1993). *Modern heuristic techniques for combinatorial problems*. John Wiley & Sons, Inc.
- Roucairol, C. (1989). *Parallel branch and bound algorithms-an overview*. PhD thesis, INRIA.
- Roucairol, C. (1996). Parallel processing for difficult combinatorial optimization problems. *European Journal of Operational Research*, 92(3):573–590.
- Samofalov and Simonenko (1972). Automatisation of university timetabling. Polytechnical Institute Publ(Russian), Kiev.
- Silver, E. A., Victor, R., Vidal, V., and de Werra, D. (1980). A tutorial on heuristic methods. *European Journal of Operational Research*, 5(3):153–162.
- Souza, M. (2022). Inteligência computacional para otimização.